

Assignment 2 - Data Structures

Dauletkyzy Bayan | SE-2523 | Design and Analysis of Algorithms

Three structures store primitive int values: a doubling dynamic array, a singly linked list with head and tail, and an array-based binary min-heap. Standard collection implementations are used only in tests. Let n denote the current number of elements.

1. Complexity of public operations

Operation	Best	Average	Worst	Auxiliary space	Reason
Array add(x)	$\Theta(1)$	$\Theta(1)^*$	$\Theta(n)$	$\Theta(n)$ on growth; else $\Theta(1)$	Doubling copies n cells only at capacity.
Array add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$ on growth; else $\Theta(1)$	Shifts $n-i$ cells; may also copy n .
Array remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Shifts $n-i-1$ cells to the left.
Array get(i)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	One indexed cell read.
Array contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Sequential scan until a match or the end.
List add(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Tail reference avoids traversal.
List add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head/end are constant; otherwise find predecessor.
List remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head is constant; otherwise find predecessor.
List get(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Follows i next links from the head.
List contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Checks nodes in sequence.
Heap insert(x)	$\Theta(1)$	$O(\log n)^{**}$	$\Theta(n)$	$\Theta(n)$ on growth; else $\Theta(1)$	Sift-up takes at most heap height; growth copies n .
Heap peekMin()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Minimum is at index zero.
Heap extractMin()	$\Theta(1)$	$\Theta(\log n)^{***}$	$\Theta(\log n)$	$\Theta(1)$	Replacement travels down at most heap height.
size() (all)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Returns a maintained field.
Heap isValidHeap()	$\Theta(1)$	$\Theta(n)^{****}$	$\Theta(n)$	$\Theta(1)$	Checks each parent-child edge on a valid heap.

* Append average means amortized over a sequence; a full-array append still costs $\Theta(n)$. Other indexed averages use a uniformly chosen valid index. Search averages assume a uniformly positioned match among distinct values, or an absent query. Best bounds apply to suitable nonempty inputs.

** Heap insertion has an amortized upper bound $O(\log n)$ and lower bound $\Omega(1)$, including doubling. Without a specified input distribution, a tight average Θ bound is not claimed. Without resizing its worst case is $\Theta(\log n)$. *** Average extraction assumes a heap obtained from random distinct keys; equal keys can stop immediately. **** Validation average considers valid heaps, as used by the tests.

The list stores $\Theta(n)$ nodes. Arrays/heaps use $\Theta(n)$ capacity while growing, but do not shrink after removals: retained space is $O(m)$, where m is the maximum size reached. Temporary growth needs another $\Theta(n)$ array; a new list node uses constant space. Θ gives matching O and Ω bounds; non-tight insertion bounds are stated separately.

2. Loop invariant proofs

Proof A: `DynamicArray.contains(value)`

Invariant. Before iteration i , $0 \leq i \leq \text{size}$ and every element at indices 0 through $i-1$ differs from `value`. The array and size are unchanged.

Initialization. i is zero, so the already checked prefix is empty. There is no element in that prefix equal to `value`.

Maintenance. The loop reads `elements[i]` and compares it with `value`. If equal, it returns `true` and a matching element has been found. Otherwise the checked prefix grows by one after $i++$, so the invariant holds for the next iteration.

Termination. If the loop ends normally, i equals `size` and the invariant covers every active element. No matching element exists, so returning `false` is correct. The variant `size-i` decreases by one on every continuing iteration, so the loop terminates.

Conclusion. A `true` return has a witnessed match; a `false` return follows a complete unsuccessful scan. This also handles an empty array because the loop executes zero times.

Proof B: `DynamicArray.remove(index)`

Let N be the old size, k the valid removal index, and A the array contents before the operation. `removedValue` is set to `A[k]` before any shifts.

Invariant. Before iteration i , $k \leq i \leq N-1$. Cells $j < k$ still equal `A[j]`; cells $k \leq j < i$ equal `A[j+1]`; cells $i \leq j < N$ still equal `A[j]`. The saved return value remains `A[k]`.

Initialization. i equals k . The shifted interval is empty and all active cells still equal their original values, so the invariant holds.

Maintenance. The condition $i < N-1$ makes $i+1$ a valid cell. By the invariant, `elements[i+1]` equals `A[i+1]`. Assigning it to `elements[i]` extends the shifted interval by one without changing the other intervals. After $i++$, the invariant is restored.

Termination. At $i=N-1$, cells k through $N-2$ contain `A[k+1]` through `A[N-1]`. The loop variant $N-1-i$ decreases to zero. Decreasing `size` to $N-1$ excludes the unused last cell.

Conclusion. The active array is the original sequence with exactly `A[k]` removed, in the same order, and the method returns `A[k]`. Removing the last element needs no shifts; an invalid index throws before any change.

3. Measurement and interpretation

For each $n = 100, 1,000, 10,000, 100,000$, input values come from `Random(42)`. Each case discards two warm-ups and records the median of five measured runs. The 36 CSV rows cover both structures in W1/W2, both structures and both variants in W3, and MinHeap in W4.

W1 performs 10,000 random gets. W2 uses 500 present and 500 negative absent queries. W3 inserts 1,000 values then removes 1,000 values at index 0 or the fixed original $n/2$. W4 inserts all n values and extracts all of them. Input preparation is outside W1-W3 timing; W4 includes both phases. Validation and CSV output are outside timing.

Counters are incremented inside methods. Steps are array-cell reads or next-link advances; moves are copied/shifted/repositioned array elements or updates of head, tail and next; comparisons involve stored values and search queries. New element placement, bounds checks and local reference assignments are excluded from moves. The validation helper does not modify metrics.

Discussion (13 sentences)

`DynamicArray` accesses index i directly, so W1 always records exactly 10,000 reads.

`MyLinkedList` follows next links, and its W1 steps grow from 501,508 to 502,489,208 as n grows from 100 to 100,000.

At $n=100,000$, W1 takes 0.0356 ms for the array and 1,241.305 ms for the list in the submitted measurements.

Contiguous int storage offers spatial locality because nearby values can share CPU cache lines.

This also benefits sequential iteration and the linear contains scan.

List nodes are separate objects, so pointer chasing makes each next address depend on the previous load.

Object headers, references and alignment add node overhead, while allocation and garbage collection can add costs; these effects were not measured separately here.

W2 therefore shows similar linear operation counts but different elapsed times: at $n=100,000$, the array takes 36.7515 ms and the list 110.5372 ms.

For W3 head operations, the list only changes a constant number of links and is the better choice for frequent front insertions and removals.

The array instead shifts elements, reaching 200,999,000 moves at $n=100,000$, while the list records 3,000 link updates.

For W3 middle operations, this singly linked list still traverses to the predecessor, so changing links cheaply does not make the complete indexed operation constant.

MinHeap is appropriate for repeatedly selecting the smallest priority, and W4 insert-plus-extract processing has a worst-case $O(n \log n)$ bound.

Decreasing tiny array or head-list timings at larger n reflect possible remaining JVM warm-up, scheduling and timing noise, so counters support the complexity claims more reliably than those small times alone.

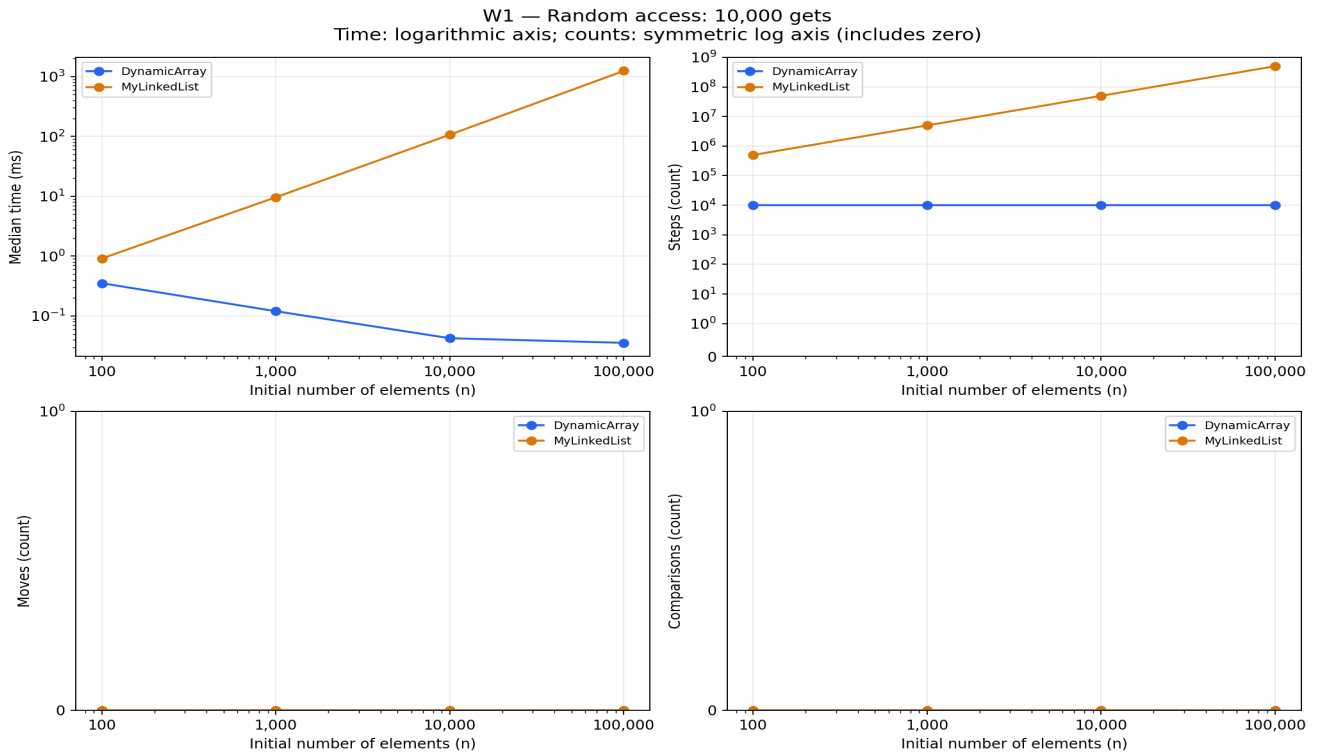
Verification and remaining submission checks

The student screenshot shows all six JUnit tests passing with exit code 0; the benchmark screenshot also shows exit code 0. The uploaded source contains randomized oracle checks, edge cases, counter checks and heap validation after mutations. The reviewer environment lacks Maven and javac, so it did not independently rerun those tests. Actual JVM flags and hardware were not recorded.

Code, counters, CSV, four plots, two proofs, operation bounds and run instructions are included. The real Git repository still needs verification: main, required feature branches, meaningful commits, tag v1.0 and a Moodle repository link. Optional JOL/buildHeap bonuses are not included.

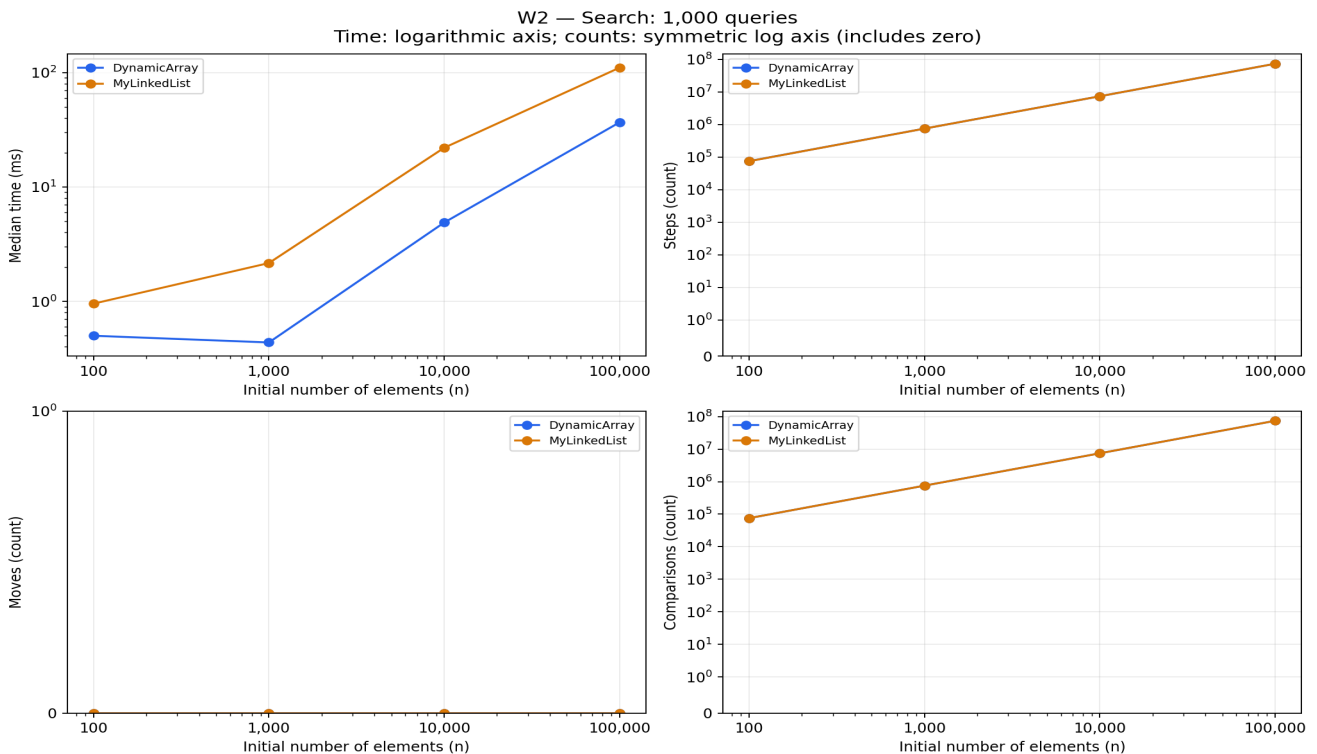
4. Workload charts: W1 and W2

W1 - Random access



Median elapsed time and measured physical operations; n is the original input size. Logarithmic axes distinguish scales; counter plots include zero values.

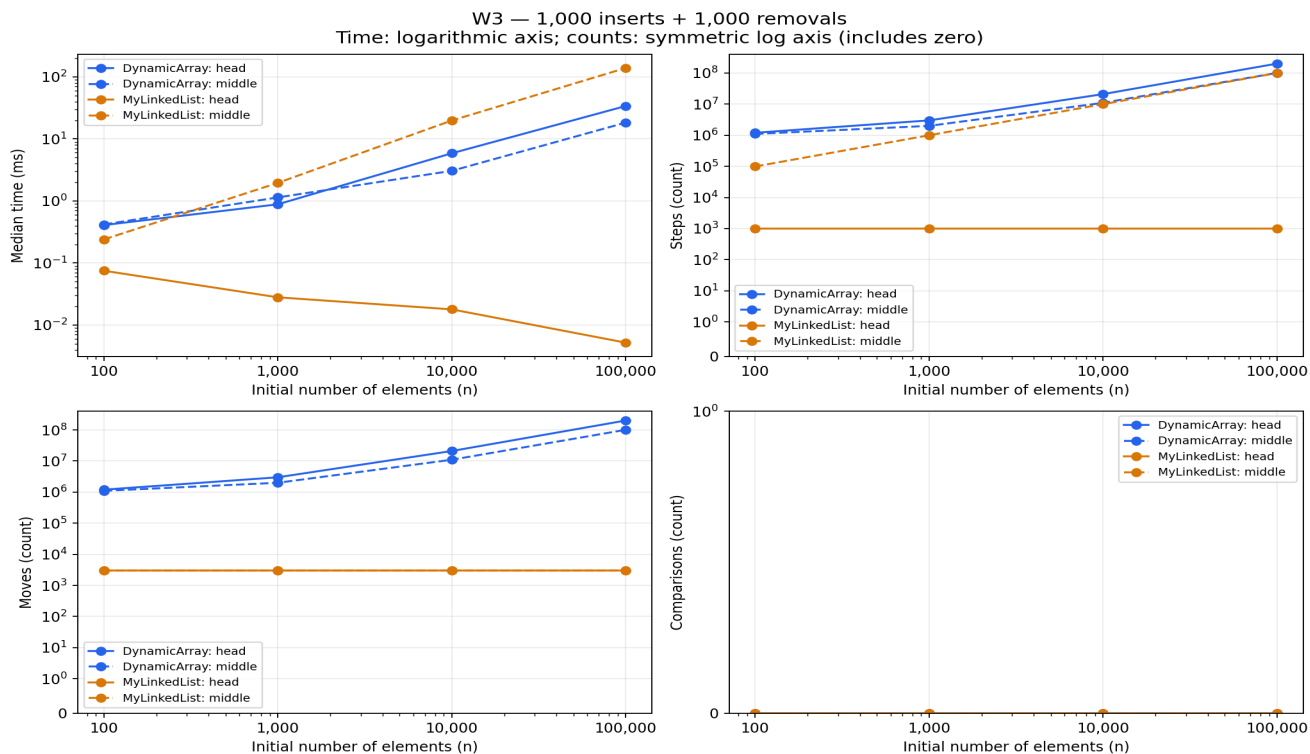
W2 - Search



Median elapsed time and measured physical operations; n is the original input size. Logarithmic axes distinguish scales; counter plots include zero values.

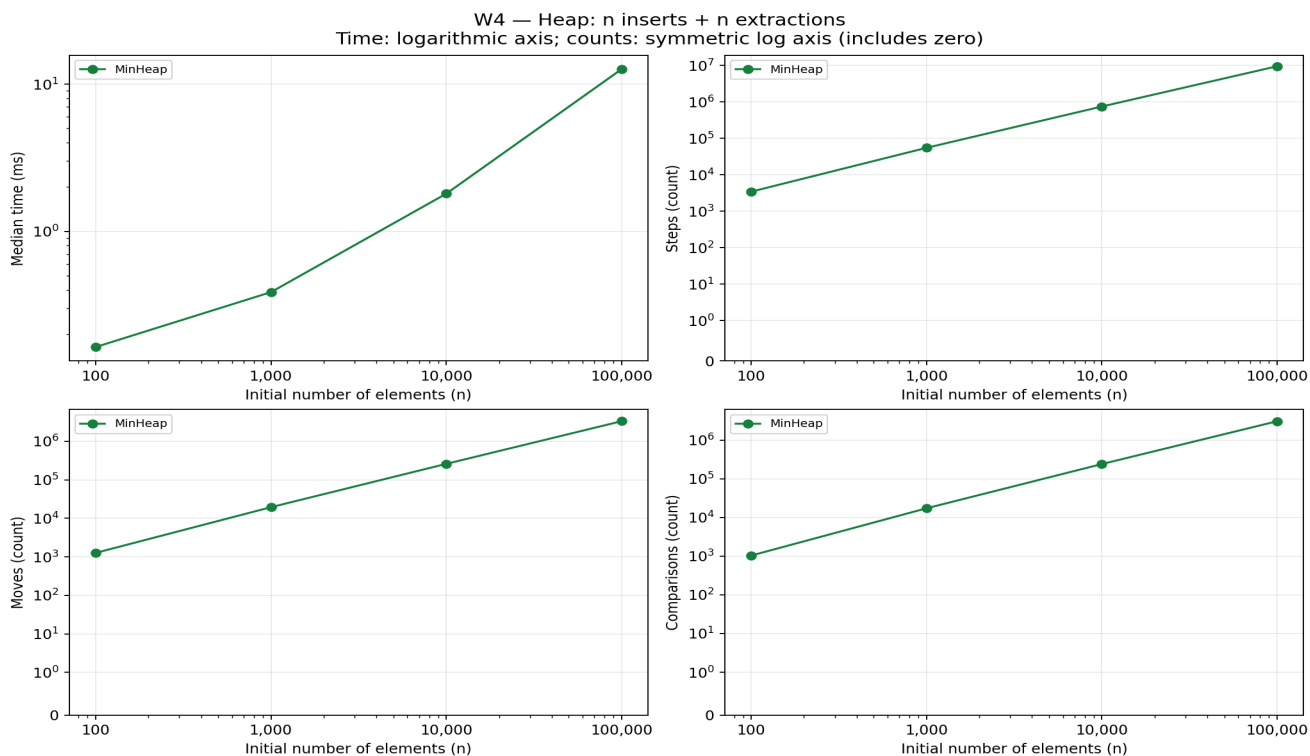
4. Workload charts: W3 and W4

W3 - Insert and remove



Median elapsed time and measured physical operations; n is the original input size. Logarithmic axes distinguish scales; counter plots include zero values.

W4 - Priority processing



Median elapsed time and measured physical operations; n is the original input size. Logarithmic axes distinguish scales; counter plots include zero values.